Checkout Service Architecture

Document date: 2024-01-15 Classification: Internal

Table of Contents

1. Overview 2
2. Data Stores 3
3. Dependencies 4
4. Security and Authentication 5
5. Scaling 6
6. Open Questions 7

1. Overview

The Checkout Service (checkout-service) is owned by the Checkout Experience team, with Daniel Okafor as technical lead. It is implemented in Node.js 20 with NestJS and listens on port 8080. Postmortems are blameless and are published within five business days. Service-level objectives are reviewed quarterly with product owners. We should revisit the alert thresholds after the next game day.

All services must expose Prometheus metrics on the /metrics path. Secrets must never be committed; the pre-commit hook scans for high-entropy strings. Staging mirrors production topology but runs at roughly one tenth of the capacity. Service-level objectives are reviewed quarterly with product owners.

2. Data Stores

Primary data store: carts cached in Redis (checkout-redis) with PostgreSQL checkout-db as the system of record. Capacity planning for the holiday peak starts in September every year. Structured JSON logging is mandatory; free-text logs are dropped by the log pipeline.

3. Dependencies

Checkout Service depends on: payment-service, inventory-service, checkout-redis, checkout-db. Staging mirrors production topology but runs at roughly one tenth of the capacity. Flaky integration tests in CI are tracked on the quality board; please do not just re-run them.

Checkout Service calls the Payment Service with a 5 second timeout and 2 retries.

4. Security and Authentication

Authentication mechanism: Checkout Service uses JWT bearer tokens validated at the api-gateway, forwarded as the X-User-Context header. Remember to tag pull requests with the owning team so reviews get routed correctly. Every service has a README that links to its dashboards and runbook.

5. Scaling

Flaky integration tests in CI are tracked on the quality board; please do not just re-run them. Container images are built once and promoted across environments by digest. Container images are built once and promoted across environments by digest. On-call handover happens every Monday at 10:00 local time. Service-level objectives are reviewed quarterly with product owners.

The dashboards were updated to show saturation alongside latency and errors. Load tests should be coordinated in the platform channel so they do not trip alerts. Database schema migrations must be backward compatible for at least one release. Flaky integration tests in CI are tracked on the quality board; please do not just re-run them.

6. Open Questions

Every service has a README that links to its dashboards and runbook. Remember to tag pull requests with the owning team so reviews get routed correctly. Database schema migrations must be backward compatible for at least one release.